چالش ۳۰ روزهٔ اندروید

آمادگی مصاحبهٔ اندروید — جونیور ← مید ← سینیور ← آرکیتکت

نسخهٔ ۲۰۲۶

روز ۲۸

عملکرد در مقیاس بزرگ — بودجهٔ عملکرد، گیت رگرسیون در CI، مانیتورینگ میدانی و پاسخ به حادثه



اسماعیل مرادی

@esmaeeil-moradi700

چرا عملکرد در مقیاس بزرگ در ۲۰۲۶ مهم است؟

روی گوشی تست خودتان، عملکرد یعنی شکار باگ. روی میلیون‌ها نصب و هزاران مدل دستگاه، عملکرد یک **سیستم** است: چه چیزی را اندازه می‌گیرید، مالکش کیست، چه چیزی جلوی merge را می‌گیرد، و وقتی ریلیزی اپ را کُند کرد چه می‌کنید.

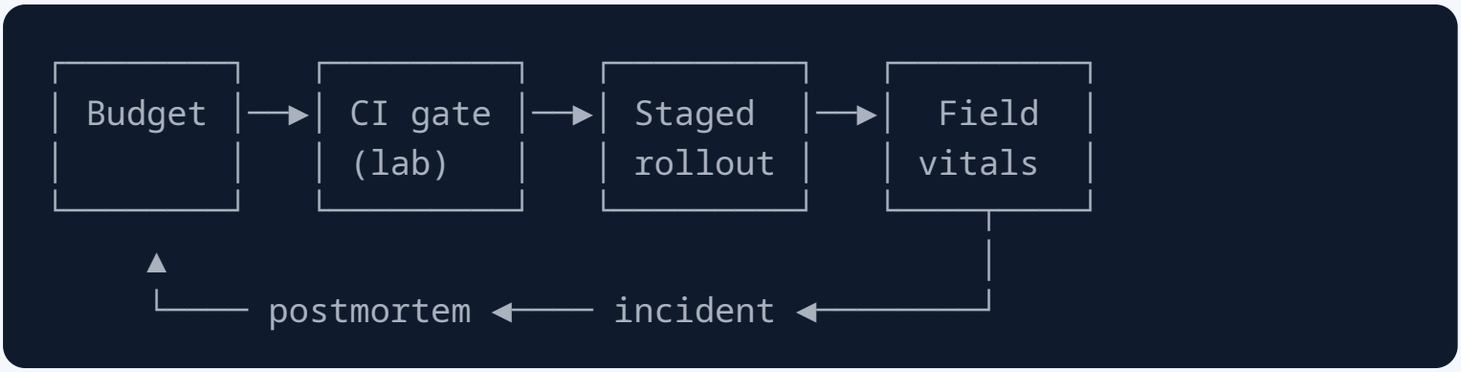
- **دُم توزیع، خودِ محصول است:** p90 و p99 روی دستگاه ضعیف، نه میانگین روی گوشی پرچم‌دار
- **lab در کنار field:** ابزار Macrobenchmark در CI رگرسیون را زودتر از کاربر می‌گیرد؛ Android vitals تجربهٔ واقعی کاربر را نشان می‌دهد
- **بودجه با مالک**، «سریع» را به عددی تبدیل می‌کند که یک نفر مسئولش است
- **Google Play اعمالش می‌کند:** عبور از آستانه‌های core vitals یعنی دیده‌شدن کمتر
- **مشکل عملکرد، حادثهٔ production است** و observability و playbook می‌خواهد

سطح	سؤال‌ها
جونیور	عملکرد در مقیاس بزرگ، lab در برابر field، متریک‌های مهم
مید	بودجه‌ها، گیت رگرسیون در CI، traceهای میدانی، رده‌بندی دستگاه‌ها
سینیور	دستگاه‌های ضعیف، observability و نمونه‌برداری، رگرسیون بعد از ریلیز
آرکیتکت	مالکیت و فرهنگ، پاسخ به حادثه، پایپ‌لاین تله‌متری



اسماعیل مرادی
@esmaeeil-moradi700

مدل ذهنی: یک چرخه، نه یک دوی سرعت



- **lab** می‌گوید *آیا یک تغییر* اپ را کُندتر کرده — کنترل‌شده، تکرارپذیر، قبل از merge
- **field** می‌گوید *کاربر چه می‌گیرد* — دستگاه، شبکه و دادهٔ واقعی، بعد از ریلیز
- **چرخه وقتی بسته می‌شود** که هر حادثه به یک محافظ جدید ختم شود: یک benchmark، یک بودجه یا یک alert

مقیاس، سؤال را عوض می‌کند

روی یک گوشی می‌پرسید «*سریع است؟*». در مقیاس بزرگ می‌پرسید «*برای چه کسی کُند است، از کدام نسخه، و کدام تیم مالکش است؟*»



اسماعیل مرادی
@esmaeeil-moradi700

جونیور

عملکرد در مقیاس بزرگ، lab در برابر field، متریک‌های مهم



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱. وقتی عملکرد «در مقیاس بزرگ» می‌شود، چه چیزی عوض می‌شود؟

روی میز شما	در مقیاس بزرگ
یک گوشی پرچم‌دار جدید	هزاران مدل، بسیاری با RAM کم
وای‌فای سریع دفتر	اینترنت موبایل کُند، captive portal، آفلاین
یک حساب تست تمیز	کاربرانی با چند سال داده و حافظهٔ پر
یک زمان‌سنجی با چشم	یک توزیع: p50، p90، p99
خودتان کُندی را می‌بینید	کسی گزارش نمی‌دهد — کاربر فقط می‌رود

مسیری کُند که فقط ۱ درصد sessionها به آن می‌خورند روی گوشی شما دیده نمی‌شود، اما با میلیون‌ها کاربر، هر روز گروه بزرگی از آدم‌ها را درگیر می‌کند.

سؤال تکمیلی

«*چرا percentile و نه میانگین؟*» — میانگین دُم توزیع را پنهان می‌کند: تعداد زیادی session سریع و چند session افتضاح، میانگینی «قابل‌قبول» می‌سازند. p90 و p99 نشان می‌دهند بدترین تجربه‌ها چه حسی دارند.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱. دو افسانه از نسخهٔ پارسال

تلهٔ مصاحبه

«*هدف ۶۰ FPS است: هر فریم در ۱۶ میلی‌ثانیه.*» — بسیاری از دستگاه‌ها با ۹۰ یا ۱۲۰ هرتز کار می‌کنند و مهلت هر فریم حدود ۱۱ یا ۸ میلی‌ثانیه است. به «از دست ندادن مهلت فریم» فکر کنید و **jank** را اندازه بگیرید، نه یک عدد ثابت FPS.

تلهٔ مصاحبه

«*برای مقیاس‌پذیری* `HashMap` *را با* `SparseArray` *عوض کنید.*» — بهینه‌سازی‌های ریز به‌ندرت متریک‌های field را تکان می‌دهند. بردهای بزرگ ساختاری‌اند: کار کمتر روی ترد اصلی در startup، allocation کمتر در مسیر اسکرول، payload کوچک‌تر. **اول اندازه بگیرید**، با ابزارهای روز ۱۷ و ۱۸.

مقیاس بزرگ واقعاً چه چیزی اضافه می‌کند

ترفند جدید profiling نه — **فرایند**: بودجه، گیت، دادهٔ field به تفکیک نسخه و دستگاه، و مالکیت روشن.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۲. دادهٔ lab در برابر دادهٔ field

	آزمایشگاه (lab)	میدان (field)
ابزارها	Macrobenchmark، Microbenchmark، Perfetto	تله‌متری خودتان، Android vitals و Firebase Performance
جواب می‌دهد	آیا این تغییر اپ را کُندتر کرد؟	کاربر واقعاً چه تجربه‌ای دارد؟
نقطهٔ قوت	تکرارپذیر، قبل از merge، trace کامل	دستگاه، شبکه و حجم دادهٔ واقعی
نقطهٔ ضعف	چند دستگاه، مسیرهای اسکریپتی	با تأخیر، پرنویز، تجمیعی، نمونه‌برداری‌شده
کاربرد	گیت رگرسیون در CI	اولویت‌بندی، سلامت ریلیز، alert

تلهٔ مصاحبه

«benchmark ما ۴۰۰ میلی‌ثانیه می‌گوید، پس کاربر هم ۴۰۰ میلی‌ثانیه می‌بیند.» — عددهای lab برای مقایسهٔ **labuildها** هستند، نه پیش‌بینی تجربهٔ کاربر. توزیع field پهن‌تر و کُندتر است و دستگاه‌هایی که شما ندارید شکلش می‌دهند.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۲. تست «در مقیاس بزرگ» سمت کلاینت

تلهٔ مصاحبه

«*اپ را با هزاران کاربر هم‌زمان load test کنید.*» — این تست **بک‌اند** است. هر دستگاه فقط یک کاربر را اجرا می‌کند. سمت کلاینت، مقیاس یعنی **تنوع**: دیتاست بزرگ، دستگاه ضعیف، شبکهٔ بد، sessionهای طولانی.

سؤال تکمیلی

«*سرعت یک تابع را چطور تست می‌کنید؟*» — با **Microbenchmark** در Jetpack و `BenchmarkRule` : گرم شدن، JIT و پایداری کلاک را مدیریت می‌کند. یک assertion با `measureTimeMillis` در تست JUnit بیشتر نویز است.

- در lab: **مسیرهای حیاتی کاربر** را benchmark کنید — startup، اسکرول فید اصلی، جستجو، پرداخت
- در field: همان مسیرها، روی دستگاه‌های واقعی و به تفکیک نسخهٔ اپ و ردهٔ دستگاه



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۳. Google Play کدام متریک‌ها را قضاوت می‌کند؟

core vitals در Android vitals و آستانه‌های رفتار بدشان:

core vital	کلی	برای هر مدل گوشی
نرخ crash از دید کاربر	۱٫۰۹ درصد	۸ درصد
نرخ ANR از دید کاربر	۰٫۴۷ درصد	۸ درصد

- نرخ یعنی سهم کاربران فعال روزانه که **در حین استفادهٔ فعال** از اپ دست‌کم یک crash یا ANR دیده‌اند
- بالاتر از آستانهٔ کلی، اپ ممکن است در Google Play کمتر نمایش داده شود. بالاتر از آستانهٔ هر مدل، ممکن است **روی همان دستگاه** کمتر نمایش داده شود و کاربر در صفحهٔ اپ در فروشگاه هشدار ببیند.
- این‌ها را هم زیر نظر بگیرید: startup کُند در حالت‌های cold، warm و hot (روز ۱)، رندر کُند، frozen frame (بیش از ۷۰۰ میلی‌ثانیه)، و wakeup و wake lock بیش از حد



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۳. متریک‌های خودتان — و تله

vitals در Play کفِ ماجراست. برای **مسیرهای حیاتی کاربر** متریک‌های سطح محصول اضافه کنید:

- **TTFD** برای هر صفحهٔ کلیدی (`reportFullyDrawn()` یا `ReportDrawnWhen`)
- **jank** هر صفحه هنگام اسکرول
- p90 **تأخیر شبکه** برای درخواست‌هایی که UI را معطل می‌کنند
- روند **حجم دانلود** و **حافظه** در هر ریلیز

تلهٔ مصاحبه

«Crashlytics می‌گوید ۹۹٫۵ درصد کاربران بدون crash هستند — پس در امانیم.» — Play، **ANR** را هم می‌شمارد، تعریف‌های خودش را دارد، و **هر مدل گوشی** را جدا قضاوت می‌کند. یک اپ می‌تواند در کل خوب باشد و روی یک گوشی ارزانِ پرفروش بالای ۸ درصد برود.



اسماعیل مرادی
@esmaeeil-moradi700

میدِ

بودجه‌ها، گیت رگرسیون در CI، traceهای میدانی، رده‌بندی دستگاه‌ها



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۴. یک بودجهٔ عملکرد طراحی کنید

بودجه یعنی **متریک**، **percentile**، ردهٔ دستگاه، آستانه و مالک.

بودجه	متریک	قاعدهٔ نمونه	چه چیزی چک می‌کند
کلد استارت	میانهٔ TTID، دستگاه ردهٔ متوسط	هیچ PR بیش از نویز اضافه نکند	در CI با Macrobenchmark
اسکرول فید	مقدار P90 برای `frameOverrunMs`	حداکثر ۰ (مهلت رعایت شود)	در CI با Macrobenchmark
حجم اپ	حجم دانلود	رشد، نیاز به تأیید دارد	در CI با `bundletool`
شبکه	بایت در هر بار لود فید	در هر ریلیز ردیابی شود	تست یکپارچه
زمان startup در field	مقدار p90 کلد استارت برای هر رده	نسخهٔ جدید بدتر از قبلی نباشد	داشبورد و alert

این قاعده‌ها فقط نمونه‌اند — عدد واقعی را از **baseline امروز** دربیاورید.

```
bundletool build-apks --bundle=app-release.aab \
  --output=app.apks
bundletool get-size total --apks=app.apks
```



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۴. بودجه‌هایی که با واقعیت دوام می‌آورند

- **بودجهٔ مطلق** (سقفی که کاربر حس می‌کند) به‌علاوهٔ **بودجهٔ نسبی** (هیچ تغییری نباید بیش از نویز اندازه‌گیری‌شده رگرسیون بیاورد)
- **ratchet:** بعد از هر بهبود، بودجه را پایین بیاورید — وگرنه فیچر بعدی بی‌صدا آن برد را خرج می‌کند
- **برای هر ردهٔ دستگاه:** دستگاه‌های ضعیف می‌توانند عدد خودشان را داشته باشند، اما همیشه خیلی پایین‌تر از خط «excessive» در Android vitals

تلهٔ مصاحبه

«*بودجهٔ startup: ۲ ثانیه روی گوشی قوی، ۵ ثانیه روی گوشی ضعیف.*» (توصیهٔ نسخهٔ پارسال) — ۵ ثانیه دقیقاً جایی است که Android vitals یک cold start را **excessive** می‌نامد. بودجه باید خیلی پایین‌تر از خط رفتار بد باشد، نه روی آن.

تلهٔ مصاحبه

«*ما بودجه داریم.*» — بدون یک مالک مشخص و alertی که به او برسد، بودجه فقط یک آرزوست.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۵. گرفتن رگرسیون در CI با Macrobenchmark

```
@RunWith(AndroidJUnit4::class)
class FeedScrollBenchmark {
    @get:Rule val rule = MacrobenchmarkRule()

    @Test fun scrollFeed() = rule.measureRepeated(
        packageName = "com.example.app",
        metrics = listOf(FrameTimingMetric()),
        compilationMode = CompilationMode.Partial(),
        startupMode = StartupMode.WARM,
        iterations = 10,
        setupBlock = { startActivityAndWait() },
    ) {
        val feed = device.findObject(By.res("feed"))
        feed.setGestureMargin(device.displayWidth / 5)
        feed.fling(Direction.DOWN)
        device.waitForIdle()
    }
}
```

- `FrameTimingMetric` مقدارهای `frameDurationCpuMs` و `frameOverrunMs` را (از API 31) به شکل P50، P90، P95 و P99 گزارش می‌دهد
- `CompilationMode.Partial()` پروفایل Baseline شما را اعمال می‌کند — همان وضعیت کدی که کاربر واقعی می‌گیرد



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۵. گیت را قابل‌اعتماد کنید

- **دستگاه واقعی، هر بار همان مدل.** امولاتور رفتار واقعی رندر و CPU را نشان نمی‌دهد؛ هر جا می‌توانید کلاک‌ها را قفل کنید.
- **هدفی شبیه release:** غیر debuggable، با R8 روشن و `profileable`. کتابخانهٔ Benchmark اجرا روی build دیباگ‌پذیر و امولاتور را خطا اعلام می‌کند — این خطاها را سرکوب نکنید.
- **توزیع‌ها را مقایسه کنید:** تعداد تکرار کافی، در مقایسه با یک baseline غلتان از `main`.
- **پله‌ها را در طول زمان تشخیص دهید**، همان کاری که CI خود Jetpack می‌کند: تغییر پایدار در چند build را علامت بزنید، نه یک جهش پرنویز.
- پایپ‌لاین: فایل JSON نتایج ← داشبورد ← کامنت روی PR.

> **تلهٔ مصاحبه**
>
> «*اگر هر متریکی ۱ درصد کُندتر شد، PR را fail کنید.*» — نویز بین اجراها اغلب از این بیشتر است. گیت‌های flaky اول بی‌صدا و بعد حذف می‌شوند. آستانه را از **واریانس اندازه‌گیری‌شده** تعیین کنید و قبل از بلاک کردن، پله را تأیید کنید.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۶. اندازه‌گیری مسیرهای حیاتی در production

```
class FirebaseTracer(
    private val perf: FirebasePerformance,
) : PerfTracer {
    override suspend fun <T> span(
        name: String,
        attrs: Map<String, String>,
        block: suspend () -> T,
    ): T {
        val trace = perf.newTrace(name)
        attrs.forEach { (k, v) -> trace.putAttribute(k, v) }
        trace.start()
        try { return block() } finally { trace.stop() }
    }
}

// Feature code only sees the PerfTracer interface
val feed = tracer.span(
    name = "feed_load",
    attrs = mapOf("tier" to tier.name),
) { repo.loadFeed() }
```

- SDK یک vendor را پشت **interface خودتان** بگذارید: بدون دست زدن به فیچرها بین Firebase، Sentry یا OpenTelemetry جابه‌جا شوید
- اسم spanها را از روی **مسیرهای کاربر** بگذارید، مثل `feed_load` و `checkout_submit`، نه از روی کلاس‌ها



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۶. jank هر صفحه با JankStats

```
// In onCreate(), after setContent { … }
PerformanceMetricsState
    .getHolderForHierarchy(window.decorView)
    .state?.putState("screen", "feed")

jankStats = JankStats.createAndTrack(window) { frame ->
    if (frame.isJank) {
        jankAggregator.record(
            durationNs = frame.frameDurationUiNanos,
            states = frame.states, // e.g. screen=feed
        )
    }
}
// onPause: jankStats.isTrackingEnabled = false
```

تلهٔ مصاحبه

«*برای هر فریم پرش‌دار یک event بفرستید.*» — یعنی هزاران event در هر session. **روی دستگاه تجمیع کنید** (شمارش و هیستوگرام برای هر صفحه) و دسته‌ای آپلود کنید.

تلهٔ مصاحبه

«*شناسهٔ کاربر را برای دیباگ تک‌تک کاربران به‌عنوان attribute اضافه کنید.*» — این PII است، و cardinality نامحدود داشبوردها را کُند و گران می‌کند. از بُعدهای **محدود** استفاده کنید: نسخه، رده، نوع شبکه، صفحه.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۷. رده‌بندی دستگاه‌ها بر اساس سخت‌افزار

```kotlin
enum class DeviceTier { LOW, MID, HIGH }

// Thresholds are examples: calibrate with your field data
fun Context.deviceTier(): DeviceTier {
    val am = getSystemService(ActivityManager::class.java)
    val mem = ActivityManager.MemoryInfo()
    am.getMemoryInfo(mem)
    val ramGb = mem.totalMem / 1_073_741_824.0
    val mpc = if (Build.VERSION.SDK_INT >= 31) {
        Build.VERSION.MEDIA_PERFORMANCE_CLASS
    } else 0
    return when {
        am.isLowRamDevice || ramGb < 3 -> DeviceTier.LOW
        mpc >= Build.VERSION_CODES.TIRAMISU -> DeviceTier.HIGH
        else -> DeviceTier.MID
    }
}
```

- `isLowRamDevice` روی دستگاه‌های کلاس Android Go مقدار `true` دارد
- **media performance class** را (از API 31) سازندهٔ دستگاه اعلام می‌کند؛ مقدار `0` یعنی «اعلام نشده»، نه «کُند». کتابخانهٔ Core Performance در Jetpack آن را به دستگاه‌های قدیمی‌تر هم می‌برد.
- یک بار محاسبه کنید، cache کنید، و **رده را به همهٔ متریک‌ها بچسبانید**



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۷. در هر رده چه چیزی را تطبیق دهیم؟

حوزه	ردهٔ ضعیف	ردهٔ قوی
تصویر	اندازهٔ decode کوچک‌تر، memory cache کوچک‌تر	رزولوشن کامل
لیست‌ها	پیش‌بارگذاری کمتر، آیتم‌های ساده‌تر	آیتم‌های غنی‌تر
انیمیشن و افکت	انیمیشن کمتر و سبک‌تر؛ بدون blur	همهٔ افکت‌ها
ویدیو	کیفیت پیش‌فرض پایین‌تر (bitrate کمتر)	کیفیت HD به‌طور پیش‌فرض
کار پس‌زمینه	تعداد کمتر jobهای هم‌زمان	عادی

تلهٔ مصاحبه

«*دستگاه‌ها را بر اساس نسخهٔ اندروید رده‌بندی کنید.*» — سطح API چیزی دربارهٔ RAM یا CPU نمی‌گوید: یک گوشی ارزان جدید، اندرویدی جدید روی سخت‌افزار ضعیف دارد. لیست‌های hard-code مدل گوشی هم کهنه می‌شوند. از **سیگنال‌های سخت‌افزاری** استفاده کنید.

تلهٔ مصاحبه

«*حالت low-end یعنی فیچر کمتر.*» — همان فیچرها، با رندر ارزان‌تر. کاربر گوشی ارزان باید بتواند همان مسیرها را تا آخر برود.



اسماعیل مرادی
@esmaeeil-moradi700

سینیور

دستگاه‌های ضعیف، observability و نمونه‌برداری، رگرسیون بعد از ریلیز



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۸. اپ را روی دستگاه‌های ضعیف زنده نگه دارید

- **حافظه گلوگاه است.** heap بزرگ‌تر یعنی کار بیشتر برای GC و kill بیشتر به‌خاطر کمبود حافظه در background — پس cold start بیشتر (روز ۱).
- **bitmap را در اندازهٔ نمایش decode کنید.** Coil و Glide وقتی اندازهٔ هدف را بدانند این کار را می‌کنند؛ هرگز یک عکس ۱۲ مگاپیکسلی را برای یک thumbnail decode نکنید.
- **پروفایل‌های Baseline بیشترین کمک را روی CPUهای کُند می‌کنند** (روز ۱ و روز ۱۸).
- **کمتر کار کنید:** مقداردهی lazy، بدون process اضافه، بدون کار پس‌زمینه‌ای که به کاربر خدمت نمی‌کند.

```
class ImageCacheTrimmer(
    private val cache: MemoryCache,
) : ComponentCallbacks2 {
    override fun onTrimMemory(level: Int) {
        // UI_HIDDEN / BACKGROUND: levels still delivered
        val hidden = ComponentCallbacks2.TRIM_MEMORY_UI_HIDDEN
        if (level >= hidden) cache.clear()
    }
    override fun onConfigurationChanged(c: Configuration) = Unit
    override fun onLowMemory() = Unit
}
// Application.onCreate(): registerComponentCallbacks(trimmer)
```



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۸. تله‌های دستگاه‌های ضعیف

تلهٔ مصاحبه

«`android:largeHeap="true"` *را فعال کنید.*» — مشکل را روی بعضی دستگاه‌ها پنهان می‌کند، روی دستگاه‌های کم‌RAM تقریباً کمکی نمی‌کند، و process شما را هدف بزرگ‌تری برای low-memory killer می‌کند. allocationها را درست کنید.

تلهٔ مصاحبه

«*به* `TRIM_MEMORY_RUNNING_CRITICAL` *واکنش نشان دهید.*» — سطح‌های `TRIM_MEMORY_RUNNING_*` از API 34 منسوخ شده‌اند و دیگر به اپ اطلاع داده نمی‌شوند. `TRIM_MEMORY_UI_HIDDEN` و `TRIM_MEMORY_BACKGROUND` را مدیریت کنید.

سؤال تکمیلی

«بدون یک کشوی پر از گوشی، دستگاه‌های ضعیف را چطور پوشش می‌دهید؟» — دست‌کم یک دستگاه ضعیف واقعی در آزمایشگاه benchmark نگه دارید، داشبوردهای field را بر اساس رده فیلتر کنید، و با `ApplicationExitInfo` و `REASON_LOW_MEMORY` ببینید هر چند وقت یک بار kill می‌شوید. امولاتور کم‌RAM برای تست functional خوب است، نه برای عدد performance.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۹. observability برای ناوگانی از دستگاه‌ها

سیگنال	جواب می‌دهد	نمونهٔ موبایل	هزینه
متریک	چقدر، چند بار؟	مقدار p90 کلد استارت به تفکیک رده	کم — تجمیعی
ردیابی (trace یا span)	زمان کجا رفت؟	در `checkout_submit`، اول API و بعد نوشتن در DB	متوسط — نمونه‌برداری‌شده
لاگ و event	اطرافش چه اتفاقی افتاد؟	نبودِ داده در cache، مقدار flagهای فعال	زیاد — نظم می‌خواهد
گزارش crash، ANR و exit	چرا مُرد؟	گزارش stack trace و `ApplicationExitInfo`	حجم کم، ارزش زیاد

- همه را **به هم وصل کنید**: همان session id، نسخهٔ اپ، ردهٔ دستگاه و مجموعهٔ flagها روی همهٔ سیگنال‌ها
- **OpenTelemetry** یک SDK اندروید دارد — گزینه‌ای مستقل از vendor برای span و متریک

تلهٔ مصاحبه

«*همه چیز را لاگ کن، بعداً فیلتر کن.*» — در موبایل، تله‌متری از باتری، اینترنت و حافظهٔ **کاربر** خرج می‌کند و شما هم برای ingestion پول می‌دهید. هر سیگنال باید به یک سؤال جواب بدهد.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۹. نمونه‌برداری بدون فریب دادن خودتان

```
data class Sample(val weight: Double, val outlier: Boolean)

class SessionSampler(private val rate: Double) {
    // One decision per session: whole journeys, not pieces
    private val inSample = Random.nextDouble() < rate

    fun decide(durationMs: Long, budgetMs: Long): Sample? =
        when {
            inSample -> Sample(1 / rate, outlier = false)
            // Kept for debugging, excluded from percentiles
            durationMs > budgetMs -> Sample(1.0, outlier = true)
            else -> null
        }
}
```

- **head sampling برای هر session** مسیرها را کامل نگه می‌دارد؛ **وزن را هم بفرستید** تا داشبورد بتواند شمارش‌ها را دوباره مقیاس کند
- **outlierهای خارج از بودجه و خطاها** را همیشه نگه دارید — اما در یک stream جدا، وگرنه percentileها را منحرف می‌کنند
- نرخ نمونه‌برداری یک **flag ریموت** است: در حادثه بالا ببریدش، وقتی هزینه زیاد شد پایین بیاوریدش



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۰. آیا این ریلیز اپ را کُندتر کرد؟

- هر متریک را به تفکیک **نسخهٔ اپ** ببینید؛ نسخهٔ جدید را با نسخهٔ قبلی **در همان روزها** و روی همان ترکیب دستگاه‌ها مقایسه کنید
- staged rollout را برای هر نسخه زیر نظر بگیرید و وقتی یک متریک اصلی از خط alert رد شد، **متوقفش کنید** (سازوکار rollout در روز ۲۴)
- متریک‌ها را با **variant فلگ‌ها و آزمایش‌ها** برچسب بزنید — خیلی از رگرسیون‌ها از طریق remote config و بدون binary جدید منتشر می‌شوند
- با traceهایی از field بازتولید کنید:

ANDROID 15+ (API 35)

`ProfilingManager` به اپ اجازه می‌دهد از دستگاه‌های کاربران واقعی system trace، heap dump یا stack sampling بخواهد، با محدودیت نرخی که سیستم اعمال می‌کند. نسخه‌های جدیدتر جمع‌آوری با trigger سیستمی را هم اضافه کرده‌اند، مثلاً حوالی یک ANR.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۰. تلهٔ cohort

تلهٔ مصاحبه

«*p90 نسخهٔ جدید از ماه پیش بدتر است، پس رگرسیون داریم.*» — کاربران اولیه یک **cohort سوگیرانه** هستند: دستگاه‌هایی که به وای‌فای وصل‌اند و شارژ می‌شوند زودتر آپدیت می‌گیرند، و یک آپدیت تازه قبل از اینکه کامپایل مبتنی بر profile به‌روز شود شروع به کار می‌کند. نسخه‌ها را **کنار هم و در یک بازهٔ زمانی** مقایسه کنید و ترکیب دستگاه‌ها را نرمال کنید.

تلهٔ مصاحبه

«*اگر میانهٔ startup در ۲۴ ساعت ۱۰ درصد بالا رفت، alert بده.*» (مثال نسخهٔ پارسال) — میانه دُم توزیع را پنهان می‌کند، و یک عدد سراسری، رگرسیونِ یک خانوادهٔ دستگاه را. روی **p90 به تفکیک نسخه و رده** alert بگذارید، با تعداد session کافی برای معنادار بودن.

سؤال تکمیلی

«*رگرسیون فقط روی دستگاه‌های یک OEM دیده می‌شود. حالا چه؟*» — vitals را به تفکیک مدل دستگاه ببینید، از همان cohort trace میدانی بگیرید و روی همان سخت‌افزار بازتولید کنید. تا وقتی درستش می‌کنید، با یک flag اثرش را برای همان cohort کم کنید.



اسماعیل مرادی
@esmaeeil-moradi700

آرکیتکت

مالکیت و فرهنگ، پاسخ به حادثه، پایپ‌لاین تله‌متری



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۱. ساختن فرهنگ عملکرد — چه کسی مالک چیست؟

لایه	سازوکار	مالک
مسیرهای حیاتی کاربر	۳ تا ۵ مسیر، هر کدام با یک بودجه	لیدهای محصول و مهندسی
بودجه و SLO	هدف برای هر مسیر و هر رده	تیم فیچری که مالک آن مسیر است
گیت‌های CI	چک حجم، Macrobenchmark و StrictMode در debug	تیم پلتفرم یا تیم عملکرد
داشبورد و alert میدانی	به تفکیک نسخه، رده و کشور	پلتفرم می‌سازد، تیم‌های فیچر پایش می‌کنند
بازبینی بعد از حادثه	یک محافظ جدید بعد از هر حادثه	تیمی که تغییر را منتشر کرده

یک **تیم پلتفرم** کوچک مالک ابزارها، گیت‌های CI، SDK تله‌متری و دانش تخصصی است. **تیم‌های فیچر مالک عددهای خودشان‌اند.**



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۱. فرهنگ، نه قهرمان‌بازی

- **عملکرد در design doc:** «این روی دستگاه ضعیف چقدر هزینه دارد؟» یک سؤال استاندارد در بازبینی است
- **ذهنیت SLO:** وقتی یک مسیر از بودجه خارج است، تیمش قبل از اضافه کردن فیچر جدید روی آن، درستش می‌کند
- **داشبوردهای قابل‌مشاهده** برای هر تیم و یک بازبینی منظم عملکرد؛ بردها را با عدد واقعی جشن بگیرید

تلهٔ مصاحبه

«*تیم عملکرد مالک عملکرد است.*» — آن‌وقت بقیه رگرسیون منتشر می‌کنند و منتظر می‌مانند آن تیم تمیزش کند. تیم مرکزی ابزار و تخصص می‌دهد؛ مالکیت پیش تیمی می‌ماند که کد را منتشر می‌کند.

تلهٔ مصاحبه

«*همهٔ متریک‌ها را روی همهٔ PRها گیت کنید.*» — نتیجه‌اش خستگی از alert است. چند متریک پایدار و پرارزش را گیت کنید؛ بقیه را **پایش** کنید.

سؤال تکمیلی

«*چطور محصول را قانع می‌کنید روی عملکرد سرمایه بگذارد؟*» — بودجه‌ها را به دادهٔ بیزینسی خودتان (conversion، retention، نظرها) و به قواعد دیده‌شدن core vitals در Play وصل کنید.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۲. یک ریلیز ANRها را بالا برده — قدم‌به‌قدم بگویید

قدم	چه می‌کنید
تشخیص	هشدار روی crash، ANR و زمان startup به تفکیک نسخه در طول rollout — نه «یک کاربر شکایت کرد»
تریاژ	دامنه را مشخص کنید: نسخه، دستگاه، OS، کشور، variant فلگ. مشکل جدید است یا قدیمی؟ شدت را تعیین کنید و یک مسئول حادثه بگذارید
مهار	توقف staged rollout؛ خاموش کردن فیچر با kill switch؛ برگرداندن تغییر سرور
رفع	انتشار hotfix با **versionCode بالاتر**، از یک مسیر سریعِ از پیش تأییدشده
یادگیری	بازبینی بدون مقصرتراشی؛ یک محافظ اضافه کنید: benchmark، تست، alert یا قاعدهٔ lint

قبل از حادثه آماده شوید

یک runbook، چرخهٔ on-call، kill switch دور فیچرهای پرریسک، داشبوردهای تفکیک‌شده بر اساس نسخه، و یک مسیر hotfix که تمرینش کرده‌اید.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۲. تله‌های حادثه

تلهٔ مصاحبه

«*ریلیز را در Play Console برگردانید (rollback).*» — متوقف کردن rollout فقط جلوی کاربران **جدید** را می‌گیرد؛ کاربرانی که آپدیت کرده‌اند همان نسخه را نگه می‌دارند، چون اندروید اپ را downgrade نمی‌کند. رفع مشکل به شکل یک ریلیز جدید با versionCode بالاتر منتشر می‌شود — پس kill switchها باید **از قبل داخل binary** باشند.

تلهٔ مصاحبه

«*مهار یعنی درست کردن باگ.*» — اول خونریزی را متوقف کنید (توقف rollout، خاموش کردن flag)، بعد با آرامش درستش کنید. hotfix عجولانه همان‌جایی است که حادثهٔ شمارهٔ دو شروع می‌شود.

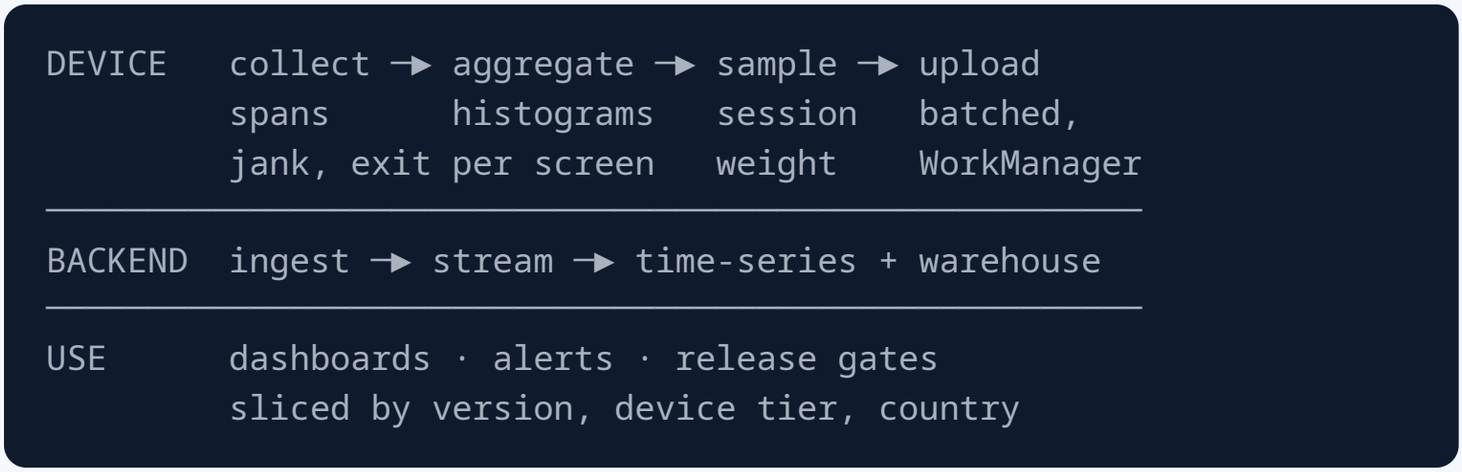
سؤال تکمیلی

«*در postmortem چه چیزی می‌آید؟*» — خط زمانی، اثر (کاربران، نسخه‌ها، مدت)، علت ریشه‌ای، اینکه چرا گیت‌ها آن را نگرفتند، و اقدام‌های مشخص با مالک و تاریخ. بدون مقصرتراشی: سیستم را درست کنید، نه آدم را.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۳. طراحی پایپ‌لاین تله‌متری برای میلیون‌ها دستگاه



- **تجمیع روی دستگاه:** به‌جای event خام، هیستوگرام برای هر متریک و هر صفحه — دادهٔ کمتر، باتری کمتر، ریسک حریم خصوصی کمتر
- **آپلود دسته‌ای** با WorkManager (روز ۱۵)، با سقف حجم و سیاست دور ریختن برای دوره‌های طولانی آفلاین
- **تله‌متری نباید خودش مشکل شود:** مقداردهی lazy بیرون از ترد اصلی، حافظه و دیسک محدود، و یک kill switch ریموت برای خود SDK
- **حریم خصوصی از روز اول:** بدون PII، با احترام به رضایت کاربر (روز ۲۹)، و اعلامش در فرم Data safety



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۳. از percentileها نمی‌شود میانگین گرفت

```
class LatencyHistogram(private val boundsMs: LongArray) {
    private val counts = LongArray(boundsMs.size + 1)

    @Synchronized fun record(ms: Long) {
        val i = boundsMs.indexOfFirst { ms <= it }
        counts[if (i == -1) boundsMs.size else i]++
    }

    // Uploaded and merged server-side by summing buckets
    @Synchronized fun snapshotAndReset(): LongArray =
        counts.copyOf().also { counts.fill(0) }
}
```

تلهٔ مصاحبه

«از p90 هر دستگاه میانگین بگیرید تا p90 کل ناوگان به دست بیاید.» — percentileها جمع‌پذیر نیستند و میانگین‌شان معنی ندارد. **هیستوگرام‌ها را ادغام کنید** (یا sketchهایی مثل t-digest و DDSketch) و percentile را از توزیع ادغام‌شده حساب کنید.

جواب سینیور در یک جمله

«عملکرد در مقیاس بزرگ یک چرخه است: برایش بودجه بگذار، در CI گیتش کن، در field به تفکیک نسخه و ردهٔ دستگاه پایشش کن، و هر حادثه را به یک محافظ جدید تبدیل کن.»



اسماعیل مرادی
@esmaeeil-moradi700

روز ۲۸ — خلاصهٔ یک‌صفحه‌ای

- دُم توزیع خودِ محصول است: **p90 و p99 به تفکیک ردهٔ دستگاه**، نه میانگین روی گوشی پرچم‌دار.
- ابزار Macrobenchmark در lab، buildها را مقایسه می‌کند؛ Android vitals در field تجربهٔ کاربر را نشان می‌دهد.
- آستانه‌های core vitals: نرخ crash **۱٫۰۹ درصد**، نرخ ANR **۰٫۴۷ درصد**، و **۸ درصد** برای هر مدل گوشی.
- بودجه یعنی متریک، percentile، رده، آستانه و **مالک**؛ بعد از هر برد پایینش بیاورید.
- گیت CI: دستگاه واقعی، build شبیه release، آستانه از روی واریانس، تشخیص پله.
- مسیرها را پشت interface خودتان trace کنید؛ attribute محدود، **بدون PII**، تجمیع روی دستگاه.
- رده‌بندی با **سیگنال سخت‌افزاری**، نه سطح API؛ همان فیچرها، رندر ارزان‌تر.
- نمونه‌برداری برای هر session همراه با وزن؛ outlierها جدا؛ **ادغام هیستوگرام**، نه میانگینِ percentile.
- ریلیزها را کنار هم و در یک بازه مقایسه کنید؛ variant فلگ‌ها را روی متریک‌ها برچسب بزنید.
- حادثه: توقف rollout، kill switch، hotfix با versionCode بالاتر، و postmortem بدون مقصرتراشی.

روز ۲۹ در راه است: مقیاس‌پذیری و انتشار جهانی. این پست را ذخیره کنید و همراه ما باشید.



اسماعیل مرادی
@esmaeeil-moradi700